

HANOI UNIVERSITY OF SCIENCE AND
TECHNOLOGY

School of Information and communications technology

Test Plan

Version 1.0

AIMS - An Internet Media Store
Subject: ITSS Software Development

Group 18

Nguyễn Tuấn Đạt – 20235907

Thành Hựu Đạt – 20235909

Lê Tùng Lâm – 20235962

Chu Tuấn Linh – 20235963

Tạ Hải Luân – 20235969

Hanoi, May 2026

Table of contents

- [1. Introduction](#)
 - [1.1. Objective](#)
 - [1.2. Scope](#)
 - [1.3. Glossary](#)
 - [1.4. References](#)
- [2. Overall Description](#)
 - [2.1. General Overview](#)
 - [2.2. Assumptions/Constraints/Risks](#)
- [3. Testing Approach/Strategy](#)
 - [3.1. Overview](#)
 - [3.2. Scope of Unit Testing](#)
 - [3.3. Strategy for designing Unit Tests](#)
 - [3.4. Tools and Frameworks](#)
- [4. Unit Testing Summary](#)
 - [4.1. Traceability from Test Cases to Use Cases](#)
 - [4.2. Test Suite for UC002-"Use Case Name"](#)
- [5. Test Case Details](#)

1. Introduction

1.1. Objective

The purpose of this document is to define the Unit Test Plan for the AIMS project developed by our team. This document describes how unit testing will be planned, designed, implemented, executed, and reported during the development phase of the project. It is intended for team members, the lecturer, teaching assistants, and any stakeholder who needs to understand how the team verifies the correctness of important software units.

This Test Plan focuses on unit testing, but does not aim to cover all testing activities of the whole system, such as integration testing, system testing, acceptance testing. Because most of the critical business rules are implemented on the backend side, the main unit testing

framework for this project is expected to be pytest with pytest-django. For frontend units, the team may use Vitest or Jest together with React Testing Library when there are suitable frontend utility functions, validators, or isolated UI logic to test.

Unit testing in this project focuses on verifying the correctness of individual software components such as functions, validators, service methods, and business logic classes. The main objectives are:

- To detect defects early before they affect larger system flows
- To verify that each selected unit behaves correctly according to the requirements
- To test important business rules such as product price validation, delivery fee calculation, cart stock checking, delivery information validation, and order cancellation/refund eligibility
- To reduce the risk of regression defects when the team updates or refactors the source code
- To provide a reliable foundation for later testing levels such as integration testing and system testing.

1.2. Scope

The software product covered by this Unit Test Plan is AIMS. AIMS is an Internet Media Store system that supports the buying, selling and management of physical media products. The system provides different functions for customers, product managers, and administrators.

This Unit Test Plan focuses on testing selected important units of the AIMS system. The selected units are mainly backend business logic units implemented in Python/Django. These units are chosen because they directly affect important user flows and contain rules that are easy to break when the system is updated.

The planned unit testing scope includes the following areas:

- Product price validation.
- Delivery fee calculation.
- Cart quantity and stock validation.
- Delivery information validation.
- Order cancellation and refund eligibility decision logic.

These units are strongly related to the AIMS requirements. For example, the AIMS problem statement specifies that product prices must remain within 30% to 150% of the original value, cart and order processing must check stock availability, delivery fees depend on weight and location, and customers may cancel orders only before approval.

The scope of this Unit Test Plan does not include full UI testing, complete end-to-end checkout testing, real payment gateway testing, real email delivery testing, production database testing, deployment testing, or performance testing. These areas may require integration or system-level testing.

The main goal of this Unit Test Plan is to make sure that the most important business logic units are tested in a clear, repeatable, and automated way. This helps the team improve code quality, reduce manual checking, and make future changes safer.

1.3. Glossary

Term	Definition
Unit Testing	A testing level that verifies individual software units such as functions, methods, validators, services, or classes in isolation
Unit	A small testable part of the software, such as a Python function, Django service method, validator, or frontend utility function
Test Plan	A document that describes the scope, strategy, tools, risks, and deliverables of testing activities
Test Case	A specific set of input data, execution conditions, expected results, and actual results used to verify a unit
Test Suite	A collection of test cases or test files that can be executed together
SUT	Software Under Test; the unit or component being tested
TDD	Test-Driven Development, a development approach where tests are written before or alongside the implementation code
Red-Green-Refactor or	The TDD cycle: write a failing test, write enough code to pass it, then refactor the code while keeping tests passing
Black-box Testing	A testing technique that designs test cases based on input and output requirements without depending on internal implementation details.
White-box Testing	A testing technique that designs test cases based on internal code structure, branches, and paths.
Equivalence Partitioning	A black-box testing technique that divides input data into groups expected to produce similar behavior
Boundary Value Analysis	A testing technique that focuses on boundary values because defects often occur near limits
Decision Table Testing	A testing technique used when several conditions combine to determine different outputs or actions
Regression Testing	Re-running existing tests after code changes to ensure previously working behavior has not been broken

pytest	A Python testing framework used to write and run automated tests
pytest-django	A pytest plugin that supports testing Django applications.
Django	A Python web framework used for the backend of the AIMS project
Vitest	A frontend testing framework often used with modern TypeScript and React projects
Jest	A JavaScript testing framework commonly used for React and Next.js projects.
React Testing Library	A testing library used to test React components from the user's perspective
Mock	A simulated object or function used to replace a real dependency during testing.
Stub	A simple replacement for a dependency that returns predefined values during testing.
Assertion	A statement in a test that checks whether the actual result matches the expected result.
Expected Result	The result that should occur if the unit behaves correctly.
Actual Result	The result observed after running the test.
VAT	Value Added Tax. In AIMS, product invoices include VAT according to the requirement.
Delivery Fee	The shipping cost calculated based on location, product weight, and free-shipping policy.
Refund Eligibility	The decision logic that determines whether an order can be refunded automatically, manually, or not at all.

1.4. References

Nguyen Thi Thu Trang. (2026). *Lab&Assignment10-UnitTesting*. School of Information and Communication Technology, Hanoi University of Science and Technology.

Nguyen Thi Thu Trang. (2026). *AIMS Problem Statement*. School of Information and Communication Technology, Hanoi University of Science and Technology.

Nguyen Thi Thu Trang. (2026). *(TEMPLATE-EN) TestPlan-UnitTest*. School of Information and Communication Technology, Hanoi University of Science and Technology.

Centers for Medicare & Medicaid Services. (n.d.). *Test Case Specification*.
<https://www.cms.gov/Research-Statistics-Data-and-Systems/CMS-Information-Technology/XLC/Downloads/TestCaseSpecification.docx>

pytest. (n.d.). *pytest Documentation*. <https://docs.pytest.org/>

pytest-django. (n.d.). *pytest-django Documentation*.
<https://pytest-django.readthedocs.io/>

Django Software Foundation. (n.d.). *Django Documentation*. <https://docs.djangoproject.com/>

Vitest. (n.d.). *Vitest Documentation*. <https://vitest.dev/>

Jest. (n.d.). *Jest Documentation*. <https://jestjs.io/docs/getting-started>

Testing Library. (n.d.). *React Testing Library Documentation*.
<https://testing-library.com/docs/react-testing-library/intro/>

2. Overall Description

2.1. General Overview

AIMS is an Internet Media Store for buying, selling, managing physical media products such as books, newspapers, CDs, and DVDs. The system is designed for three main user groups: customers, product managers, and administrators.

Customers can browse products, search by title or category, filter products by price range, view product details, add products to cart, update cart quantities, provide delivery information, place orders, make payments, and cancel orders if the order has not yet been approved. Product managers can manage product information, update stock, view product history, and process pending orders. Administrators can manage user accounts, assign roles, and perform account-related administrative actions.

The project is implemented using a modern web application architecture. The frontend is developed with TypeScript, Next.js, and React. The backend is developed with Python and Django. The database layer uses PostgreSQL, managed through Supabase. This separation allows the frontend to focus on user interaction, while the backend handles business logic, data validation, order processing, and database-related operations.

From a testing perspective, the most important unit testing targets are the backend business logic units. These units are responsible for enforcing core rules of the system, such as valid product price ranges, delivery fee formulas, stock availability, delivery information formats, and order cancellation/refund conditions. If these units are incorrect, customers may see wrong prices, invalid orders may be accepted, shipping fees may be calculated incorrectly, or refund behavior may be handled incorrectly. Therefore, this Unit Test Plan focuses mainly on backend unit tests using pytest and pytest-django, meanwhile frontend

unit tests using Vitest or Jest may be added if the team has isolated frontend logic that can be tested without relying on backend integration.

No new high-level context diagram is introduced in this Test Plan. The unit testing work is based on the existing AIMS requirements, current Django backend structure, and the team's chosen implementation approach. If the system design or source code structure changes significantly, this Test Plan will be reviewed and updated.

2.2. Assumptions/Constraints/Risks

2.2.1. Assumptions

- The project uses TypeScript, Next.js, and React for the frontend; Python and Django for the backend; and PostgreSQL/Supabase for the database
- The main unit testing framework for backend logic is pytest, with pytest-django used when Django-specific behavior needs to be tested; for frontend will be Vitest or Jest together with React Testing Library
- Test data will be mock data only and will not include real customer information, real payment data, real passwords, or sensitive credentials
- External services such as PayPal, VietQR, email services, and production Supabase services will not be called directly during unit testing.

2.2.2. Constraints

- Scope constraint: Test Plan only covers unit testing. It does not cover full integration testing, end-to-end testing, system testing, acceptance testing, performance testing, or complete UI testing
- The testing tools must match the selected technology stack. Backend tests should use Python-based testing tools such as pytest and pytest-django. Frontend tests, if implemented, should use tools suitable for TypeScript/Next.js/React, such as Vitest or Jest.
- Real payment gateways, QR payment APIs, email services, and third-party systems should not be called during unit tests. These dependencies should be mocked, stubbed, or excluded from unit testing scope.
- Environment constraint: Test execution may be affected by differences in Python version, Node.js version, package versions, operating systems, environment variables, or local configuration files
- Some existing code may be difficult to test if business logic is tightly coupled with Django views, database queries, API calls, or UI handling. In such cases, the team may need to refactor the code into services, validators, or utility functions before writing tests.
- Team members may have different levels of experience with pytest, Django testing, frontend testing, mocking, and test case design. This may affect test quality and consistency.

2.2.3. Risks

Risk Description	Mitigation Strategy
The team chooses units that are too simple, such as basic getters, setters, or display-only UI components	Select business logic units such as price validation, delivery fee calculation, stock validation, delivery info validation, and refund eligibility
Test cases do not cover enough valid, invalid, and boundary cases	Apply equivalence partitioning, boundary value analysis, and decision table testing when designing test cases
Unit tests accidentally become integration tests by calling the real database, Supabase, or external APIs.	Keep tests focused on pure functions and service logic. Mock database/API dependencies where possible.
Backend code is too tightly coupled with Django views or database models, making unit testing difficult.	Refactor business logic into separate service or validator functions before writing tests.
Different team members use inconsistent testing styles or naming conventions	Agree on common test naming rules, file structure, and test case format before implementation
Tests pass on one member's machine but fail on another machine	Standardize Python, Node.js, dependency versions, and provide clear test execution commands
Business rules are misunderstood, causing incorrect expected results in test cases	Cross-check expected results against the AIMS problem statement and discuss unclear rules within the team before finalizing test cases
External payment behavior such as PayPal or VietQR refund cannot be tested reliably in unit tests	Test only internal refund decision logic. Do not call real PayPal, VietQR, or production payment APIs
Test methods become too large and difficult to understand.	Keep each test focused on one behavior and use parameterized tests for similar input-output cases.

3. Testing Approach/Strategy

3.1. Overview

The unit testing approach for this project is designed to verify the correctness of important business logic units in the AIMS system. Since AIMS includes many rules related to product pricing, cart validation, delivery calculation, order processing, and refund

handling, the team will focus on units that are small enough to test independently but important enough to affect the correctness of the whole system.

The team follows a practical unit testing approach based on the following principles:

- Test-Driven Development: We think about test cases before or during implementation. In some cases, tests may be written after the initial code is available, but the expected behavior must still be derived from the requirements rather than from the current implementation.
- Behavior-focused testing: Each unit test should verify a specific behavior of the unit. For example, a delivery fee calculation unit should be tested against different locations, weights, and order values according to the business rules.
- Isolation of units: Unit tests should avoid depending on real Supabase data, production databases, external payment gateways, or email services. When a dependency is required, the team will use mock objects, stubs, dummy data, or isolated test data.
- Regression-oriented testing: The selected units are business-critical and likely to be affected by future changes. Therefore, the unit tests are also designed to work as regression tests to make sure that previously correct behavior is not broken after updates.

3.2. Scope of Unit Testing

The scope of unit testing in this project is limited to individual software units that can be tested independently. The team will mainly test backend functions, validators, and services because these units contain the core business rules of the AIMS system.

- In scope

Unit ID	Unit / Function Area	Description	Main Framework
U001	Product Price Validation	Validate whether the current price of a product is within the allowed range of 30% to 150% of the original value	pytest
U002	Delivery Fee Calculation	Calculate shipping fee based on delivery location, total weight, order value, and free-shipping policy	pytest
U003	Cart Quantity / Stock Validation	Check whether the requested quantity in the cart is valid and whether stock is sufficient	pytest

U004	Delivery Information Validation	Validate customer name, phone number, and delivery address before placing an order	pytest / Vitest if frontend validation is included
U005	Order Cancellation / Refund Eligibility	Determine whether an order can be cancelled and whether refund should be automatic or manual	pytest

- Out of scope
 - + Complete frontend-to-backend user flows
 - + End-to-end checkout process
 - + Real Supabase production database operations
 - + Real PayPal or VietQR payment gateway calls
 - + Real email sending
 - + UI layout, styling, responsiveness, or visual rendering
 - + Performance testing
 - + Security penetration testing
 - + Full integration testing between multiple modules

3.3. Strategy for designing Unit Tests

The unit test design strategy for the AIMS project is mainly based on black-box testing, with white-box testing used as a supplementary strategy. Our team prioritizes black-box testing because the selected units are business logic functions whose expected behavior can be derived directly from the AIMS problem statement. Test cases are designed from input conditions, expected outputs, validation rules, and business decisions rather than from the implementation only.

For each unit, our team first identifies the input domain, valid and invalid input classes, important boundary values, and combinations of conditions that affect the output. After the test cases are designed from requirements, the implementation is reviewed to make sure that important branches and statements are executed by the test suite.

The following black-box test design techniques are used:

A. Equivalence Partitioning

Equivalence Partitioning is used to divide input data into groups that are expected to produce the same type of result. Instead of testing every possible input value, our team selects representative values from each equivalence class. This helps reduce the number of test cases while still covering important behaviors.

This technique is applied to the following units:

- Product Price Validation:

The input domain is divided into valid price range, price below the allowed range, price above the allowed range, invalid original value, and invalid current price.

- Delivery Fee Calculation:

The input domain is divided into major city provinces, other provinces, valid weight, invalid weight, free-shipping eligible orders, non-free-shipping orders, valid province input, and invalid province input.

- Cart Quantity / Stock Validation:

The input domain is divided into valid quantity, insufficient stock, invalid requested quantity, and invalid stock quantity.

- Delivery Information Validation:

The input domain is divided into valid and invalid customer names, valid and invalid phone numbers, valid and invalid addresses, and missing-field cases.

- Order Cancellation / Refund Eligibility:

The input domain is divided into cancellable status, non-cancellable status, valid payment method, invalid payment method, automatic refund case, and manual refund case.

B. Boundary Value Analysis

Boundary Value Analysis is used because defects often occur at the edge of input ranges or near comparison conditions. Our team selects values exactly at the boundary, just below the boundary, and just above the boundary where applicable.

This technique is applied to the following units:

- Product Price Validation:

Boundary values include 30% of the original value, just below 30%, 150% of the original value, and just above 150%.

- Delivery Fee Calculation:

Boundary values include 3kg for Ha Noi and Ho Chi Minh City, 3.5kg for the next 0.5kg unit, 0.5kg for other provinces, 1.0kg for the next 0.5kg unit, order value exactly 100,000 VND, order value greater than 100,000 VND, and weight equal to 0.

- Cart Quantity / Stock Validation:

Boundary values include requested quantity equal to stock quantity, requested quantity greater than stock quantity by one, requested quantity equal to 0, and stock quantity equal to 0.

- Delivery Information Validation:

Boundary values include customer name length 30 and 31, phone number length 9, 10, and 11 digits, and address length 100 and 101.

C. Decision Table Testing

Decision Table Testing is used for units where multiple conditions combine to determine different outputs. Our team identifies the conditions, possible values of each condition, and the expected action for each meaningful combination.

This technique is applied to the following units:

- **Delivery Fee Calculation:**

The main decision conditions are delivery province type, weight range, and free-shipping eligibility. The output changes depending on whether the province is a major city or another province, whether the weight is within the base weight or exceeds it, and whether the order value is greater than 100,000 VND.

- **Delivery Information Validation:**

The phone number validation logic depends on whether the phone number has no separator, one separator type, multiple separator types, starts with 0, and contains exactly 10 digits after separators are removed.

- **Order Cancellation / Refund Eligibility:**

The refund result depends on order status and payment method. If the order is pending processing and paid by PayPal, the result is an automatic refund. If the order is pending processing and paid by VietQR, the result is manual refund required. If the order is already approved, rejected, or cancelled, the customer cannot cancel it for refund. Invalid status and invalid payment methods are also tested.

D. Use Case-Based Testing

Use Case-Based Testing is used to ensure that the selected unit tests support important AIMS use cases. Each selected unit is traced to one or more use cases in the Unit Testing Summary section.

- Product Price Validation supports product management use cases.
- Cart Quantity / Stock Validation supports viewing cart and placing order use cases.
- Delivery Fee Calculation supports placing order and entering delivery information use cases.
- Delivery Information Validation supports entering delivery information and placing order use cases.
- Order Cancellation / Refund Eligibility supports cancel order and refund use cases.

White-box testing is used as a supplementary strategy after black-box test cases are created. Our team reviews the source code of each unit and checks whether important statements and branches are executed by the test suite.

The following white-box techniques are considered:

- **Statement Coverage**

Our team checks whether each important statement in the target function is executed by at least one test case. For example, delivery fee tests execute the statements for province normalization, major city base fee calculation, other province base fee

calculation, extra weight fee calculation, free-shipping discount, and invalid input exceptions.

- **Branch Coverage**

Our team checks whether both true and false outcomes of important conditions are covered. For example:

- + In `calculate_delivery_fee()`, tests cover valid province and invalid province, major city and other province, weight within base weight and weight above base weight, free-shipping eligible and non-free-shipping orders.
- + In `validate_cart_item_quantity()`, tests cover invalid requested quantity, invalid stock quantity, insufficient stock, and valid quantity.
- + In delivery information validators, tests cover valid and invalid branches for name, phone number, and address.
- + In `determine_refund_action()`, tests cover invalid status, invalid payment method, non-refundable status, PayPal automatic refund, and VietQR manual refund.

Path Coverage is not selected as the main target because full path coverage may create too many combinations for some units. Instead, our team focuses on requirement-based black-box test cases and enough white-box branch coverage to make sure that important decision paths are executed.

The general test design process is as follows:

1. Read the related AIMS requirement and identify the unit under test.
2. Identify inputs, outputs, preconditions, and validation rules.
3. Design test cases using equivalence partitioning, boundary value analysis, and decision table testing.
4. Write automated pytest test cases with clear test IDs and expected outputs.
5. Run the tests and compare actual results with expected results.
6. Review the source code to check whether important statements and branches are covered.
7. Refactor the code if necessary while keeping all tests passing.
8. Save test execution evidence as screenshots for the UnitTesting/Images folder.

3.4. Tools and Frameworks

Our team uses modern testing tools that match the technology stack of the AIMS project. Since most selected units are backend business logic units, the main testing tools are Python, Django, pytest, and pytest-django.

A. Python

Python is used for backend implementation and backend unit testing. The selected backend units are implemented as Python functions in service or validator modules. Python is suitable

for this project because the backend of AIMS is developed with Django, and the selected business rules can be tested as isolated Python functions.

For money-related and weight-related calculations, Decimal is used instead of float when appropriate. This helps avoid floating-point precision problems in units such as product price validation and delivery fee calculation.

B. Django

Django is the backend web framework of the AIMS project. Some selected units belong to Django apps such as products, carts, and orders. However, the unit tests are designed to focus on service and validator functions instead of complete Django views or API flows.

Our team avoids testing complete frontend-to-backend-to-database flows in unit tests. If a unit depends on Django models or database queries, the dependency should be isolated, mocked, or moved into a separate service function when possible.

C. pytest

pytest is the main backend unit testing framework used by our team. It is selected because it is simple, readable, and suitable for testing Python functions, services, and validators. pytest allows our team to write concise test cases and run them automatically from the command line.

Our team uses pytest features such as:

- assert statements to compare actual outputs with expected outputs
- `pytest.mark.parametrize` to define multiple input-output test cases clearly
- `pytest.raises` to verify that invalid inputs raise the expected exceptions
- verbose mode (`-v`) to display clear test case names and support screenshot evidence

D. unittest.mock / pytest monkeypatch

Mocking tools may be used when a unit depends on external services, database repositories, payment gateways, email services, or other modules. The purpose of mocking is to isolate the unit under test and avoid calling real external dependencies.

Examples of dependencies that should be mocked or excluded from unit tests include:

- Supabase or production database calls
- PayPal API calls
- VietQR API calls
- Email sending services
- External HTTP requests

For the current delivery fee calculation unit, mocking is not required because `calculate_delivery_fee()` is a pure function. It receives input values and returns a calculated fee without database access or API calls.

E. `coverage.py`

`coverage.py` may be used as an optional tool to measure how much of the source code is executed by the unit tests. It helps our team review statement coverage and identify code branches that are not tested yet.

4. Unit Testing Summary

4.1. Traceability from Test Cases to Use Cases

Use Case ID	Use Case Name
UC001	Manage Product
UC002	Place Order
UC003	Enter Delivery Information
UC004	Cancel Order /Refund

USE CASE ID			UC001	UC002	UC003	UC004
Test Case ID	Test Case Title	Totals	15	20	47	8
CSV_001	Validate quantity within stock	1		x		
CSV_002	Validate quantity equals stock	1		x		
CSV_003	Reject quantity above stock	1		x		
CSV_004	Reject zero requested quantity	1		x		

CSV_005	Reject negative requested quantity	1		x		
CSV_006	Report missing quantity when stock is zero	1		x		
CSV_007	Reject negative stock quantity	1		x		
PPV_001	EP: Current price within valid range (50% of original)	1	x			
PPV_002	Current price is too low (20% of original)	1	x			
PPV_003	Current price is too high (200% of original)	1	x			
PPV_004	Current price is just below 30% (29,999)	1	x			
PPV_005	Current price is exactly 30% boundary	1	x			
PPV_006	Current price is just above 30% (30,001)	1	x			
PPV_007	Current price is just below 150% (149,999)	1	x			

PPV_008	Current price is exactly 150% boundary	1	x			
PPV_009	Current price is just above 150% (150,001)	1	x			
PPV_010	Original value is 0	1	x			
PPV_011	Original value is negative (< 0)	1	x			
PPV_012	Current price is negative (< 0)	1	x			
PPV_013	Original value is None	1	x			
PPV_014	Current price is None	1	x			
PPV_015	Massive scale value stress test (50,000,000)	1	x			
DIV_001	Validate normal customer name	1			x	
DIV_002	Validate customer name with outer spaces	1			x	

DIV_003	Validate customer name at maximum length	1			x	
DIV_004	Reject missing customer name	1			x	
DIV_005	Reject empty customer name	1			x	
DIV_006	Reject blank customer name	1			x	
DIV_007	Reject customer name over maximum length	1			x	
DIV_008	Reject customer name containing numbers	1			x	
DIV_009	Reject customer name containing special characters	1			x	
DIV_010	Validate normal phone number with 10 digits	1			x	
DIV_011	Validate phone number with dash separator	1			x	
DIV_012	Validate phone number with dot separator	1			x	
DIV_013	Validate phone number with slash separator	1			x	
DIV_014	Validate phone number with outer spaces	1			x	
DIV_015	Reject missing phone number	1			x	
DIV_016	Reject empty phone number	1			x	

DIV_01 7	Reject phone number not starting with 0	1			x	
DIV_01 8	Reject phone number with only 9 digits	1			x	
DIV_01 9	Reject phone number with 11 digits	1			x	
DIV_02 0	Reject phone number with mixed separators	1			x	
DIV_02 1	Reject phone number with repeated separators	1			x	
DIV_02 2	Reject phone number with spaces inside	1			x	
DIV_02 3	Reject phone number containing letters	1			x	
DIV_02 4	Validate normal delivery address	1			x	
DIV_02 5	Validate address containing letters and numbers	1			x	
DIV_02 6	Validate address at maximum length	1			x	
DIV_02 7	Reject missing address	1			x	
DIV_02 8	Reject empty address	1			x	
DIV_02 9	Reject blank address	1			x	
DIV_03 0	Reject address over maximum length	1			x	

DIV_03 1	Reject address containing unsupported special character	1			x	
DIV_03 2	Validate full delivery information with all valid fields	2		x	x	
DIV_03 3	Return errors for full delivery information with invalid fields	2		x	x	
DIV_03 4	Return errors for full delivery information with missing fields	2		x	x	
DFC_0 01	Calculate delivery fee for Ha Noi within base weight	2		x	x	
DFC_0 02	Calculate delivery fee for Ha Noi with extra 0.5kg	2		x	x	
DFC_0 03	Calculate delivery fee for Ho Chi Minh City within base weight	2		x	x	
DFC_0 04	Calculate delivery fee for other province within base weight	2		x	x	
DFC_0 05	Calculate delivery fee for other province with extra 0.5kg	2		x	x	
DFC_0 06	Apply free-shipping discount for major city order	2		x	x	

DFC_007	Apply free-shipping discount for other province order	2		x	x	
DFC_008	Reject empty province input	2		x	x	
DFC_009	Reject zero weight input	2		x	x	
DFC_010	Reject negative order value input	2		x	x	
DFC_011	Normalize province before checking major city	2		x	x	
DFC_012	Do not apply free-shipping discount at exact threshold	2		x	x	
DFC_013	Round extra weight up to next 0.5kg unit	2		x	x	
DFC_014	Reject blank province input	2		x	x	
OCR_001	PENDING_PROCESSING status can cancel order	1				x
OCR_002	APPROVED status cannot cancel order	1				x
OCR_003	REJECTED status cannot cancel order	1				x
OCR_004	CANCELLED status cannot cancel order	1				x
OCR_005	PENDING + PAYPAL returns AUTO_REFUND	1				x

OCR_006	PENDING + VIETQR returns MANUAL_REFUND_REQUIRED	1				x
OCR_007	APPROVED + PAYPAL returns NOT_REFUNDABLE_BY_CUSTOMER_CANCEL	1				x
OCR_008	Invalid order status returns INVALID_ORDER_STATUS	1				x
OCR_009	Invalid payment method returns INVALID_PAYMENT_METHOD	1				x

4.2. Test Suite for UC001 - "Manage Product"

Test Suite ID	Test Suite Title	Description	Test Cases
US001	Product Price Validation	This test suite verifies the validation logic for product pricing within the product management module. It ensures that a product's current price is strictly within the permitted range of 30% to 150% of its original value. The suite comprehensively covers equivalence partitions (valid and invalid price ranges), boundary values (exact 30% and 150% thresholds), guard clauses (handling of zero and negative values), and system	PPV_001, PPV_002, PPV_003, PPV_004, PPV_005, PPV_006, PPV_007, PPV_008, PPV_009, PPV_010, PPV_011, PPV_012, PPV_013, PPV_014, PPV_015

		edge/stress cases (None values and massive scale thresholds) based on the test implementation.	
--	--	--	--

4.3. Test Suite for UC002 - "Place order"

Test Suite ID	Test Suite Title	Description	Test Cases
US001	Cart quantity / stock validation	Verify cart item quantity rules for valid quantity, insufficient stock, invalid requested quantity, and invalid stock quantity.	CSV_001, CSV_002, CSV_003, CSV_004, CSV_005, CSV_006, CSV_007
US002	Delivery Fee Calculation Test Suite	This test suite verifies the delivery fee calculation logic used during the order placing flow. It checks whether the system correctly calculates shipping fees based on delivery province, total item weight, order value, and the free-shipping policy. The suite includes major city cases, other province cases, base-weight boundary cases, extra-weight rounding cases, free-shipping discount cases, and invalid input cases.	DFC_001, DFC_002, DFC_003, DFC_004, DFC_005, DFC_006, DFC_007, DFC_008, DFC_009, DFC_010, DFC_011, DFC_012, DFC_013, DFC_014

4.4. Test Suite for UC003 - "Enter Delivery Information"

Test Suite ID	Test Suite Title	Description	Test Cases
---------------	------------------	-------------	------------

US004	Delivery Information Validation Test Suite	This test suite verifies the validation logic for customer delivery information before an order can proceed. It covers customer name, phone number, address, and the combined delivery information object. The suite includes valid cases, invalid cases, missing-field cases, and boundary cases for maximum length and phone format.	DIV_001, DIV_002, DIV_003, DIV_004, DIV_005, DIV_006, DIV_007, DIV_008, DIV_009, DIV_010, DIV_011, DIV_012, DIV_013, DIV_014, DIV_015, DIV_016, DIV_017, DIV_018, DIV_019, DIV_020, DIV_021, DIV_022, DIV_023, DIV_024, DIV_025, DIV_026, DIV_027, DIV_028, DIV_029, DIV_030, DIV_031, DIV_032, DIV_033, DIV_034
-------	--	--	--

4.5. Test Suite for UC004 - "Cancel Order /Refund"

Test Suite ID	Test Suite Title	Description	Test Cases
US005	Order Cancellation / Refund Eligibility Test Suite	This test suite verifies the order cancellation and refund eligibility logic in the cancel order/refund use case. It checks whether customers are allowed to cancel orders based on order status and whether the system correctly determines the refund action according to the payment method. The suite includes cancellable and non-cancellable order states, automatic refund cases, manual refund cases, invalid order status cases, and invalid payment method cases.	OCR_001, OCR_002, OCR_003, OCR_004, OCR_005, OCR_006, OCR_007, OCR_008, OCR_009

5. Test Case Details

Test Case ID	Test Case Name	Description	Unit Under Test	Input Data	Expected Output	Actual Output	Pass/Fail	Notes

PPV_001	EP: Current price within valid range (50% of original)	Verify that a current price at exactly 50% of the original value is accepted.	validate_product_price()	original_value=100000, current_price=50000	Valid	Valid	Pass	Equivalence partitioning: valid range
PPV_002	EP: Current price is too low (20% of original)	Verify that a current price below 30% of the original value is rejected.	validate_product_price()	original_value=100000, current_price=20000	Invalid	Invalid	Pass	Equivalence partitioning: invalid too low
PPV_003	EP: Current price is too high (200% of original)	Verify that a current price above 150% of the original value is rejected.	validate_product_price()	original_value=100000, current_price=200000	Invalid	Invalid	Pass	Equivalence partitioning: invalid too high
PPV_004	BVA: Current price is just below 30% (29,999)	Verify that a current price just 1 unit below the 30% threshold is rejected.	validate_product_price()	original_value=100000, current_price=29999	Invalid	Invalid	Pass	Boundary value: lower bound - 1
PPV_005	BVA: Current price is exactly	Verify that a current price exactly at the 30%	validate_product_price()	original_value=100000, current	Valid	Valid	Pass	Boundary value:

	30% boundary	lower limit is accepted.		t_price=30000				exact lower bound
PPV_006	BVA: Current price is just above 30% (30,001)	Verify that a current price just 1 unit above the 30% threshold is accepted.	validate_product_price()	original_value=100000, current_price=30001	Valid	Valid	Pass	Boundary value: lower bound + 1
PPV_007	BVA: Current price is just below 150% (149,999)	Verify that a current price just 1 unit below the 150% threshold is accepted.	validate_product_price()	original_value=100000, current_price=149999	Valid	Valid	Pass	Boundary value: upper bound - 1
PPV_008	BVA: Current price is exactly 150% boundary	Verify that a current price exactly at the 150% upper limit is accepted.	validate_product_price()	original_value=100000, current_price=150000	Valid	Valid	Pass	Boundary value: exact upper bound
PPV_009	BVA: Current price is just above 150% (150,001)	Verify that a current price just 1 unit above the 150% threshold is rejected.	validate_product_price()	original_value=100000, current_price=150001	Invalid	Invalid	Pass	Boundary value: upper bound + 1

PPV_010	Guard: Original value is 0	Verify that an original value of 0 is rejected to prevent division by zero.	validate_product_price()	original_value=0, current_price=10000	Invalid	Invalid	Pass	Guard clause: zero value check
PPV_011	Guard: Original value is negative (< 0)	Verify that a negative original value is treated as invalid and rejected.	validate_product_price()	original_value=-10000, current_price=50000	Invalid	Invalid	Pass	Guard clause: negative input check
PPV_012	Guard: Current price is negative (< 0)	Verify that a negative current price is treated as invalid and rejected.	validate_product_price()	original_value=10000, current_price=-1	Invalid	Invalid	Pass	Guard clause: negative input check
PPV_013	Edge: Original value is None	Verify that a missing (None) original value is safely handled and rejected.	validate_product_price()	original_value=None, current_price=50000	Invalid	Invalid	Pass	Edge case: null handling
PPV_014	Edge: Current price is None	Verify that a missing (None) current price is safely	validate_product_price()	original_value=10000, current	Invalid	Invalid	Pass	Edge case: null handling

		handled and rejected.		t_price =None				
PPV_015	Edge: Massive scale value stress test (50,000,000)	Verify that the function can correctly process large currency figures without overflowing .	validate_product_price()	original_value=50000000, current_price=50000000	Valid	Valid	Pass	Stress test: large scale values

- Unit: Cart quantity / stock validation

Test Case ID	Test Case Name	Description	Unit Under Test	Input Data	Expected Output	Actual Output	Pass/Fail	Notes
CSV_001	Validate quantity within stock	Requested quantity is lower than available stock	apps.carts.services.validate_cart_item_quantity	requested=1, stock=10	{valid: True, reason: None, missing_quantity: 0}	{valid: True, reason: None, missing_quantity: 0}	Pass	Equivalence partitioning: valid quantity
CSV_002	Validate quantity equals stock	Requested quantity exactly equals available stock	apps.carts.services.validate_cart_item_quantity	requested=10, stock=10	{valid: True, reason: None, missing_quantity: 0}	{valid: True, reason: None, missing_quantity: 0}	Pass	Boundary value: requested = stock
CSV_003	Reject quantity above stock	Requested quantity exceeds stock by one item	apps.carts.services.validate_cart_item_quantity	requested=11, stock=10	{valid: False, reason: INSUFFICIENT_STOCK, missing_quantity: 1}	{valid: False, reason: INSUFFICIENT_STOCK, missing_quantity: 1}	Pass	Boundary value: requested = stock + 1

CSV_004	Reject zero requested quantity	Requested quantity is zero	apps.carts.services.validate_cart_item_quantity	requested=0, stock=10	{valid: False, reason: INVALID_QUANTITY, missing_quantity: 0}	{valid: False, reason: INVALID_QUANTITY, missing_quantity: 0}	Pass	Boundary value: requested = 0
CSV_005	Reject negative requested quantity	Requested quantity is negative	apps.carts.services.validate_cart_item_quantity	requested=-1, stock=10	{valid: False, reason: INVALID_QUANTITY, missing_quantity: 0}	{valid: False, reason: INVALID_QUANTITY, missing_quantity: 0}	Pass	Equivalence partitioning: invalid requested quantity
CSV_006	Report missing quantity when stock is zero	No stock is available for a positive request	apps.carts.services.validate_cart_item_quantity	requested=5, stock=0	{valid: False, reason: INSUFFICIENT_STOCK, missing_quantity: 5}	{valid: False, reason: INSUFFICIENT_STOCK, missing_quantity: 5}	Pass	Boundary value: stock = 0
CSV_007	Reject negative stock quantity	Stock quantity is invalid because it is negative	apps.carts.services.validate_cart_item_quantity	requested=5, stock=-1	{valid: False, reason: INVALID_STOCK, missing_quantity: 0}	{valid: False, reason: INVALID_STOCK, missing_quantity: 0}	Pass	Equivalence partitioning: invalid stock quantity

- Unit: Delivery information validation

Test Case ID	Test Case Name	Description	Unit Under Test	Input Data	Expected Output	Actual Output	Pass /Fail	Notes
--------------	----------------	-------------	-----------------	------------	-----------------	---------------	------------	-------

DIV_001	Valid normal customer name	Verify that a normal alphabetic customer name is accepted.	validate_customer_name()	"Nguyen Van A"	True	True	Pass	Equivalence partitioning: valid name
DIV_002	Valid customer name with outer spaces	Verify that leading and trailing spaces are trimmed.	validate_customer_name()	" Tran Thi B "	True	True	Pass	Valid input with trimming
DIV_003	Valid customer name at max length	Verify that a name with exactly 30 characters is accepted.	validate_customer_name()	"A" * 30	True	True	Pass	Boundary value: length = 30
DIV_004	Invalid missing customer name	Verify that a missing name is rejected.	validate_customer_name()	None	False	False	Pass	Required field validation
DIV_005	Invalid empty customer name	Verify that an empty name is rejected.	validate_customer_name()	""	False	False	Pass	Invalid empty input
DIV_006	Invalid blank customer name	Verify that a name containing only spaces is rejected.	validate_customer_name()	" "	False	False	Pass	Invalid blank input
DIV_007	Invalid customer name over max length	Verify that a name longer than 30 characters is rejected.	validate_customer_name()	"A" * 31	False	False	Pass	Boundary value: length = 31
DIV_008	Invalid customer name with number	Verify that a name containing digits is rejected.	validate_customer_name()	"Nguyen123"	False	False	Pass	Invalid character class

DIV_009	Invalid customer name with special character	Verify that a name containing special characters is rejected.	validate_customer_name()	"Nguyen@Van"	False	False	Pass	Invalid special character
DIV_010	Valid phone number with 10 digits	Verify that a 10-digit phone number starting with 0 is accepted.	validate_phone_number()	"0123456789"	True	True	Pass	Valid phone format
DIV_011	Valid phone number with dash separator	Verify that a phone number using one dash separator type is accepted.	validate_phone_number()	"0123-456-789"	True	True	Pass	Valid separator rule
DIV_012	Valid phone number with dot separator	Verify that a phone number using one dot separator type is accepted.	validate_phone_number()	"0123.456.789"	True	True	Pass	Valid separator rule
DIV_013	Valid phone number with slash separator	Verify that a phone number using one slash separator type is accepted.	validate_phone_number()	"0123/456/789"	True	True	Pass	Valid separator rule

DIV_01_4	Valid phone number with outer spaces	Verify that leading and trailing spaces in phone number are trimmed.	validate_phone_number()	" 0123456789 "	True	True	Pass	Valid input with trimming
DIV_01_5	Invalid missing phone number	Verify that a missing phone number is rejected.	validate_phone_number()	None	False	False	Pass	Required field validation
DIV_01_6	Invalid empty phone number	Verify that an empty phone number is rejected.	validate_phone_number()	""	False	False	Pass	Invalid empty input
DIV_01_7	Invalid phone number not starting with 0	Verify that a phone number not starting with 0 is rejected.	validate_phone_number()	"1234567890"	False	False	Pass	Format rule: must start with 0
DIV_01_8	Invalid phone number with 9 digits	Verify that a phone number with fewer than 10 digits is rejected.	validate_phone_number()	"012345678"	False	False	Pass	Boundary value: 9 digits
DIV_01_9	Invalid phone number with 11 digits	Verify that a phone number with more than 10 digits is rejected.	validate_phone_number()	"01234567890 "	False	False	Pass	Boundary value: 11 digits

DIV _02 0	Invalid phone number with mixed separators	Verify that a phone number using more than one separator type is rejected.	validate_phone_number()	"0123-456/789"	False	False	Pass	Decision table: mixed separator
DIV _02 1	Invalid phone number with repeated separator	Verify that repeated separators creating empty parts are rejected.	validate_phone_number()	"0123--456789"	False	False	Pass	Invalid separator placement
DIV _02 2	Invalid phone number with spaces inside	Verify that spaces inside the phone number are rejected.	validate_phone_number()	"0123 456 789"	False	False	Pass	Unsupported separator
DIV _02 3	Invalid phone number containing letters	Verify that a phone number containing alphabetic characters is rejected.	validate_phone_number()	"01234abc89"	False	False	Pass	Invalid character class
DIV _02 4	Valid normal address	Verify that a normal address with slash and letters is accepted.	validate_address()	"12/5 ABC Street"	True	True	Pass	Valid address
DIV _02 5	Valid address with letters and numbers	Verify that an address containing letters and numbers is accepted.	validate_address()	"No 10 Nguyen Trai"	True	True	Pass	Valid address characters

DIV_02_6	Valid address at max length	Verify that an address with exactly 100 characters is accepted.	validate_address()	"A" * 100	True	True	Pass	Boundary value: length = 100
DIV_02_7	Invalid missing address	Verify that a missing address is rejected.	validate_address()	None	False	False	Pass	Required field validation
DIV_02_8	Invalid empty address	Verify that an empty address is rejected.	validate_address()	""	False	False	Pass	Invalid empty input
DIV_02_9	Invalid blank address	Verify that an address containing only spaces is rejected.	validate_address()	" "	False	False	Pass	Invalid blank input
DIV_03_0	Invalid address over max length	Verify that an address longer than 100 characters is rejected.	validate_address()	"A" * 101	False	False	Pass	Boundary value: length = 101
DIV_03_1	Invalid address with unsupported special character	Verify that an address containing unsupported special characters is rejected.	validate_address()	"12@ABC"	False	False	Pass	Invalid special character
DIV_03_2	Valid full delivery information	Verify that complete delivery information with valid fields is accepted.	validate_delivery_info()	{ "name": "Nguyen Van A", "phone": "0123-456-789", "address": "12/5 ABC Street" }	{ "valid": True, "errors": {} }	{ "valid": True, "errors": {} }	Pass	Combined valid case

DIV_033	Invalid full delivery information	Verify that invalid name, phone, and address return field-level errors.	validate_delivery_info()	{ "name": "Nguyen123", "phone": "1234567890", "address": "12@ABC"} }	{ "valid": False, "errors": { "name": "INVALID_CUSTOMER_NAME", "phone": "INVALID_PHONE_NUMBER", "address": "INVALID_ADDRESS"} }	Same as expected	Pass	Combined invalid case
DIV_034	Missing full delivery information fields	Verify that missing name, phone, and address return field-level errors.	validate_delivery_info()	{ }	{ "valid": False, "errors": { "name": "INVALID_CUSTOMER_NAME", "phone": "INVALID_PHONE_NUMBER", "address": "INVALID_ADDRESS"} }	Same as expected	Pass	Missing-field validation

- Unit: Delivery Fee Calculation

Test Case ID	Test Case Name	Description	Unit Under Test	Input Data	Expected Output	Actual Output	Pass/Fail	Notes
--------------	----------------	-------------	-----------------	------------	-----------------	---------------	-----------	-------

DFC_001	Calculate Ha Noi base delivery fee	Customer is in Ha Noi and total weight is exactly within the first 3kg base range.	apps.order s.services.calculate_delivery_fee	province="Ha Noi", weight_kg=Decimal("3.0") , order_value=Decimal("90000")	22000	22000	Pass	Boundary value: major city base weight = 3kg
DFC_002	Calculate Ha Noi delivery fee with extra weight	Customer is in Ha Noi and total weight exceeds the first 3kg by 0.5kg.	apps.order s.services.calculate_delivery_fee	province="Ha Noi", weight_kg=Decimal("3.5") , order_value=Decimal("90000")	24500	24500	Pass	Boundary value: major city base weight + 0.5kg
DFC_003	Calculate Ho Chi Minh City base delivery fee	Customer is in Ho Chi Minh City and total weight is exactly within the first 3kg base range.	apps.order s.services.calculate_delivery_fee	province="Ho Chi Minh City", weight_kg=Decimal("3.0") , order_value=Decimal("90000")	22000	22000	Pass	Equivalence partitioning: major city

DFC_004	Calculate other province base delivery fee	Customer is in another province and total weight is exactly within the first 0.5kg base range.	apps.orders.services.calculate_delivery_fee	province="Da Nang", weight_kg=Decimal("0.5"), order_value=Decimal("90000")	30000	30000	Pass	Boundary value: other province base weight = 0.5kg
DFC_005	Calculate other province delivery fee with extra weight	Customer is in another province and total weight exceeds the first 0.5kg by 0.5kg.	apps.orders.services.calculate_delivery_fee	province="Da Nang", weight_kg=Decimal("1.0"), order_value=Decimal("90000")	32500	32500	Pass	Boundary value: other province base weight + 0.5kg
DFC_006	Apply free-shipping discount for Ha Noi order	Customer is in Ha Noi, total weight is within base range, and order value is greater than 100,000 VND.	apps.orders.services.calculate_delivery_fee	province="Ha Noi", weight_kg=Decimal("3.0"), order_value=Decimal("150000")	0	0	Pass	Decision table: major city + free-shipping eligible

DFC_007	Apply free-shipping discount for other province order	Customer is in another province, total weight includes extra weight, and order value is greater than 100,000 VND.	apps.orders.services.calculate_delivery_fee	province="Da Nang", weight_kg=Decimal("1.0"), order_value=Decimal("150000")	7500	7500	Pass	Decision table: other province + free-shipping eligible
DFC_008	Reject empty province input	Province is an empty string, so the delivery fee cannot be calculated.	apps.orders.services.calculate_delivery_fee	province="", weight_kg=Decimal("1.0"), order_value=Decimal("90000")	Raise ValueError	Raise ValueError	Pass	Equivalence partitioning: invalid province
DFC_009	Reject zero weight input	Total weight is zero, which is invalid for delivery fee calculation.	apps.orders.services.calculate_delivery_fee	province="Ha Noi", weight_kg=Decimal("0"), order_value=Decimal("90000")	Raise ValueError	Raise ValueError	Pass	Boundary value: weight = 0

DFC_010	Reject negative order value input	Order value is negative, which is invalid for free-shipping and delivery fee calculation.	apps.order s.services.calculate_delivery_fee	province="H a Noi", weight_kg=Decimal ("1.0") , order_value=Decimal ("-1")	Raise ValueError	Raise ValueError	Pass	Equivalence partitioning: invalid order value
DFC_011	Normalize province before checking major city	Province has leading and trailing spaces and lowercase spelling but should still be recognized as Ha Noi.	apps.order s.services.calculate_delivery_fee	province="hanoi", weight_kg=Decimal ("3.0") , order_value=Decimal ("90000")	22000	22000	Pass	Equivalence partitioning: normalized major city name
DFC_012	Do not apply discount at exact threshold	Order value is exactly 100,000 VND, so free-shipping discount must not be applied because the rule requires value greater than 100,000 VND.	apps.order s.services.calculate_delivery_fee	province="H a Noi", weight_kg=Decimal ("3.0") , order_value=Decimal ("100000")	22000	22000	Pass	Boundary value: order value = 100,000

DFC_013	Round extra weight up to next 0.5kg unit	Customer is in another province and total weight exceeds 0.5kg by only 0.1kg, so the extra weight should be rounded up to one 0.5kg unit.	apps.orders.services.calculate_delivery_fee	province="Da Nang", weight_kg=Decimal("0.6"), order_value=Decimal("90000")	32500	32500	Pass	Boundary value: extra weight rounded up
DFC_014	Reject blank province input	Province contains only spaces, so it is treated as invalid input.	apps.orders.services.calculate_delivery_fee	province=" ", weight_kg=Decimal("1.0"), order_value=Decimal("90000")	Raise ValueError	Raise ValueError	Pass	Equivalence partitioning: blank province

-Unit: Order Cancellation

Test Case ID	Test Case Name	Description	Unit Under Test	Input Data	Expected Output	Actual Output	Pass/Fail	Notes
OCR_001	PENDING_PROCESSING status can cancel order	Verify that an order with PENDING_PROCESSING status can be cancelled by the customer.	apps.orders.services.determine_refund_action	order_status="PENDING_PROCESSING", payment_method	CANCELLABLE	CANCELLABLE	Pass	Valid cancellable status

				= "PAYPAL"				
OCR_002	APPROVED status cannot cancel order	Verify that an APPROVED order cannot be cancelled by the customer.	apps.orders.service.determine_refund_action	order_status="APPROVED", payment_method="PAYPAL"	NOT_REFUNDABLE_BY_CUSTOMER_CANCEL	NOT_REFUNDABLE_BY_CUSTOMER_CANCEL	Pass	Non-cancellable status
OCR_003	REJECTED status cannot cancel order	Verify that a REJECTED order cannot be cancelled by the customer.	apps.orders.service.determine_refund_action	order_status="REJECTED", payment_method="PAYPAL"	NOT_REFUNDABLE_BY_CUSTOMER_CANCEL	NOT_REFUNDABLE_BY_CUSTOMER_CANCEL	Pass	Non-cancellable status
OCR_004	CANCELLED status cannot cancel order	Verify that an already CANCELLED order cannot be cancelled again.	apps.orders.service.determine_refund_action	order_status="CANCELLED", payment_method="PAYPAL"	NOT_REFUNDABLE_BY_CUSTOMER_CANCEL	NOT_REFUNDABLE_BY_CUSTOMER_CANCEL	Pass	Already cancelled order
OCR_005	PENDING + PAYPAL returns AUTO_REFUND	Verify that a pending order paid via PayPal returns automatic refund action.	apps.orders.service.determine_refund_action	order_status="PENDING_PROCESSING", payment_method	AUTO_REFUND	AUTO_REFUND	Pass	Decision table: PayPal automatic refund

				= "PAYPAL"				
OCR_006	PENDING + VIETQR returns MANUAL_REFUND_REQUIRED	Verify that a pending order paid via VietQR requires manual refund processing.	apps.orders.services.determine_refund_action	order_status="PENDING_PROCESSING", payment_method="VIETQR"	MANUAL_REFUND_REQUIRED	MANUAL_REFUND_REQUIRED	Pass	Decision table: VietQR manual refund
OCR_007	APPROVED + PAYPAL returns NOT_REFUNDABLE_BY_CUSTOMER_CANCEL	Verify that approved orders are not refundable even if payment method is PayPal.	apps.orders.services.determine_refund_action	order_status="APPROVED", payment_method="PAYPAL"	NOT_REFUNDABLE_BY_CUSTOMER_CANCEL	NOT_REFUNDABLE_BY_CUSTOMER_CANCEL	Pass	Approved order restriction
OCR_008	Invalid order status returns INVALID_ORDER_STATUS	Verify that an unsupported order status is rejected.	apps.orders.services.determine_refund_action	order_status="UNKNOWN_STATUS", payment_method="PAYPAL"	INVALID_ORDER_STATUS	INVALID_ORDER_STATUS	Pass	Invalid input validation
OCR_009	Invalid payment method returns INVALID_PAYMENT	Verify that an unsupported payment method is rejected.	apps.orders.services.determine_refund_action	order_status="PENDING_PROCESSING"	INVALID_PAYMENT_METHOD	INVALID_PAYMENT_METHOD	Pass	Invalid payment method validation

	_METHOD			, payment_method ="CASH"				
--	---------	--	--	--------------------------------	--	--	--	--